Test Project

<Mobile Applications Development>

JP2026 July Online Friendly Competition

JP26JulyOFC

Module A(KR) - Phone

(2.5 hours)

Contents

Contents **2**
Introduction **4**
Description of project and tasks **4**
1. General Requirements 6
1. Description *6*
2. Common Layout & Tab Navigation 7
1. Description: *7*
2. Requirements *7*
3. Security Tab – Hash Generator 9
1. Description: *9*
2. Requirements *9*
4. Security Tab – File Hash Calculator 11
1. Description: *11*
2. Requirements *11*
5. Security Tab – AES Encryption 13
1. Description: *13*
2. Requirements *13*
6. Security Tab – HMAC Generator 15
1. Description: *15*
2. Requirements *15*
7. Encoding Tab – Base64 & URL Encoder/Decoder 16
1. Description: *16*
2. Requirements *16*
8. Encoding Tab – Number Base & ASCII/HEX Converter 18
1. Description: *18*
2. Requirements *18*
9. Network Tab – CIDR & IP Address Calculator 20
1. Description: *20*
2. Requirements *20*
10. Network Tab – Port Checker (Wireframe) 22
1. Description: *22*
2. Requirements *22*

11. Tools Tab – Password & UUID Generator 23
1. Description: *23*
2. Requirements *23*
12. Tools Tab – Regex Tester & Random Number Generator 25
1. Description: *25*
2. Requirements *25*
Instructions to the Competitor **27**

Introduction

IT Utility Toolkit is a smartphone application that provides a collection of practical utilities for IT developers and security engineers.

The application offers fifteen tools across four categories: Security, Encoding, Network, and Tools. Each tool performs real calculations such as hash generation, symmetric encryption, data encoding, and network address calculation.

For this task, competitors must implement a tab-based utility application according to the provided prototypes and resource files. All tools must perform their calculations locally inside the application, and every action must provide clear success or error feedback to the user.

The purpose of this task is to evaluate the competitor's ability to build a form-based mobile UI, implement input validation and user feedback, integrate cryptographic and encoding functions, and perform bitwise network calculations within a limited competition time.

Description of project and tasks

Competitors must use the provided prototypes, icons, and test data files to implement all requirements specified in this Test Project.

The task consists of one smartphone application with four tabs (Security, Encoding, Network, Tools). The application contains fourteen functional utilities and one wireframe-only Port Checker screen. All fifteen tool screens must be implemented.

The application does not require any external API or server communication. All calculations must be performed locally on the device. The Port Checker tool is provided as a wireframe UI only and does not require real network communication.

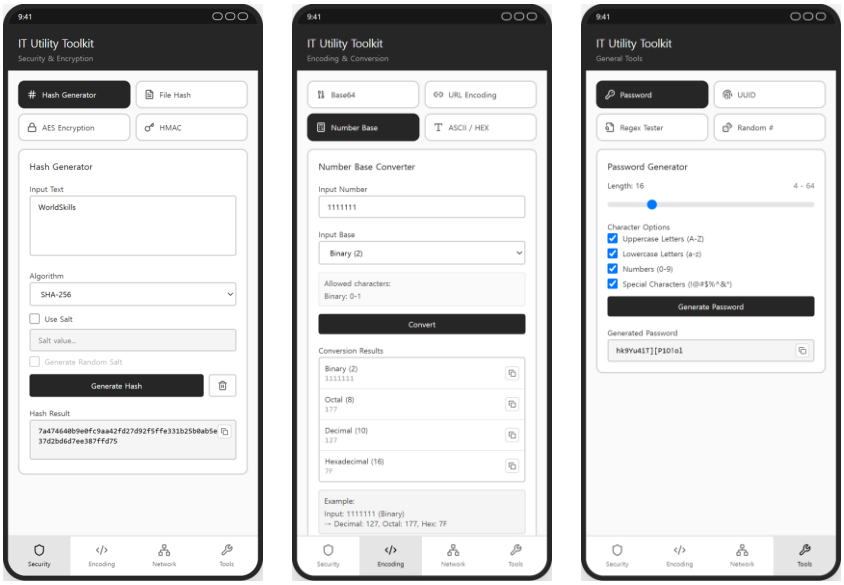
Unless stated otherwise, all text is converted to bytes using UTF-8, hash and HMAC results are displayed as lowercase hexadecimal strings, and Base64 output uses the standard alphabet without line breaks.

The provided test-vectors.json file contains the expected outputs and the common processing rules (encoding, letter case, Base64 variant, AES key/IV) for the sample inputs shown in the prototypes. It will be used for marking and may also be used by competitors to verify their implementation.

The final application will be evaluated on the provided smartphone device or smartphone emulator.

iOS: iOS 26 for Apple iPhone 17 (Simulator)

Android: Android 16 (API 36) for Pixel 9 (Emulator)


9:41
IT Utility Toolkit
Security & Encryption
Hash Generator
File Hash
AES Encryption
HMAC
Hash Generator
Input Text
WorldSkills
Algorithm
SHA-256
Use Salt
Salt value...
Generate Random Salt
Generate Hash
Hash Result
7a474640b9e0fc9aa42fd27d92f5ffe331b25b0ab5e
37d2bd6d7ee387ffd75
Security
Encoding
Network
Tools
9:41
IT Utility Toolkit
Encoding & Conversion
Base64
URL Encoding
Number Base
ASCII / HEX
Number Base Converter
Input Number
1111111
Input Base
Binary (2)
Allowed characters:
Binary: 0-1
Convert
Conversion Results
Binary (2)
1111111
Octal (8)
177
Decimal (10)
127
Hexadecimal (16)
7F
Example:
Input: 1111111 (Binary)
→ Decimal: 127, Octal: 177, Hex: 7F
Security
Encoding
Network
Tools
9:41
IT Utility Toolkit
General Tools
Password
UUID
Regex Tester
Random #
Password Generator
Length: 16
4 - 64
Character Options
Uppercase Letters (A-Z)
Lowercase Letters (a-z)
Numbers (0-9)
Special Characters (!@#$%^&*)
Generate Password
Generated Password
hk9Yu4iT][P1Ol
Security
Encoding
Network
Tools

<Prototype>

1. General Requirements

1. Description

1.1 Competitors must refer to the provided prototypes and resource files, and implement all required functionalities accordingly.

1.2 This task does not require any external API or server communication. All calculations must be performed locally inside the application.

1.3 Competitors may use only the platform SDK libraries and the packages included in the provided offline library cache. (Kotlin/Android: Java Cryptography Architecture – MessageDigest, Mac, Cipher, java.util.UUID / Swift/iOS: CryptoKit, CommonCrypto / Flutter: crypto, uuid — the offline cache contains no AES or file-picker package, so AES and file selection must be implemented in pure Dart or through a platform channel.)

1.4 Unless stated otherwise, all text is converted to bytes using UTF-8. Hash and HMAC results must be displayed as lowercase hexadecimal strings.

1.5 All success and error feedback must be displayed as an in-app notification banner (toast-style) that appears at the top of the screen and disappears automatically.

1.6 If there are undefined or missing elements in the prototypes, competitors must implement them following standard mobile application UI/UX principles.

1.7 Competitors may use all files included in the "media-files" folder.

1.8 Friendly and user-centered messages must be used for error handling, guidance, and validation.

1.9 The final application will be evaluated on the provided Android emulator or iOS Simulator and must function correctly in that environment.

1.10 No user account or authentication mechanism is required for this task.

1.11 The application must support the phone portrait layout.

1.12 The application must display the application name "IT Utility Toolkit" in the operating system.

1.13 When the application starts, the Security tab must be selected by default and the Hash Generator tool must be displayed.

1.14 The visual design must follow the grayscale (neutral) design tone shown in the provided prototypes.

2. Common Layout & Tab Navigation

1. Description:

1.1 The application consists of a fixed header area, a scrollable content area, and a fixed bottom tab navigation with four tabs.

1.2 Each tab contains a tool selector grid at the top and displays one active tool card below it.

2. Requirements

ELEMENTS

[Header Area]

1. Application title text: "IT Utility Toolkit"
2. Header subtitle text (changes according to the active tab)

[Content Area]

3. Scrollable tool content area

[Tool Selector Area]

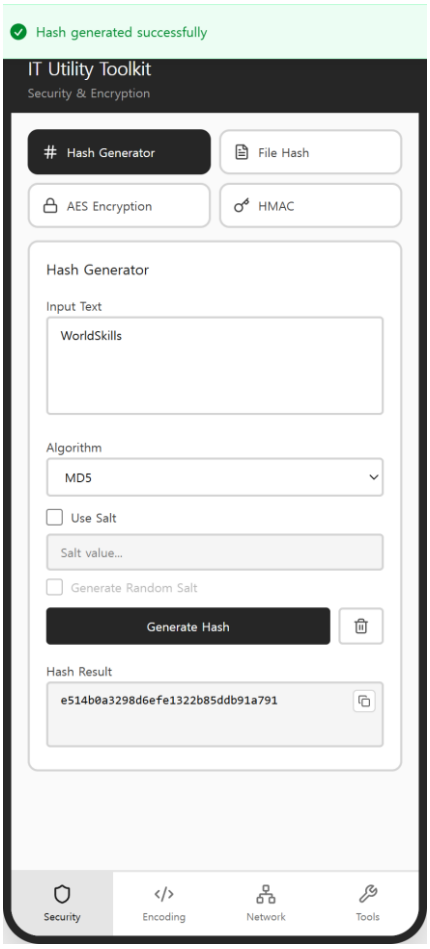
4. Tool selector button grid (2×2 or 1×3) at the top of each tab
5. Tool selector button: icon & label

[Bottom Tab Navigation]

6. Tab item: "Security" & shield icon
7. Tab item: "Encoding" & code icon
8. Tab item: "Network" & network icon
9. Tab item: "Tools" & wrench icon

[Notification Area]

10. Success / error in-app notification banner (toast-style)



REQUIREMENTS

1. When the application starts, the Security tab must be selected and the Hash Generator tool must be displayed.
2. The active tab item must be visually highlighted in the bottom tab navigation.

3. The header subtitle must change according to the active tab: "Security & Encryption", "Encoding & Conversion", "Network Utilities", "General Tools". 4. The header and the bottom tab navigation must remain fixed while the content area scrolls. 5. Selecting a tool in the tool selector must switch the displayed tool card. 6. The selected tool button must be visually highlighted. 7. A success notification banner must be displayed after a calculation, conversion, generation, file selection, or clipboard copy completes successfully. A failed validation must display an error notification banner. 8. Tab changes, tool selection, slider changes, checkbox changes, and input focus changes do not require success notifications. 9. Notification banners must appear at the top of the screen and disappear automatically. 10. Copy buttons must be displayed only when a result exists. 11. Tapping a Copy button must copy the result to the clipboard and display a "Copied to clipboard" notification banner.	

3. Security Tab – Hash Generator

1. Description:

1.1 The Hash Generator creates a hash value from an input text using the selected algorithm.

1.2 An optional salt value can be appended to the input text before hashing.

2. Requirements

ELEMENTS

[Input Area]

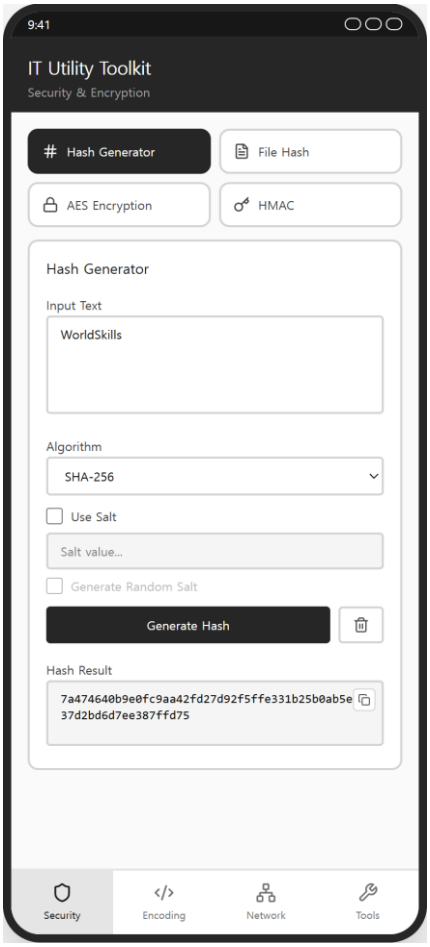
1. Input text area (placeholder: "Enter text to hash...")
2. Algorithm dropdown: MD5, SHA-1, SHA-224, SHA-256, SHA-384, SHA-512
3. Checkbox: "Use Salt"
4. Salt value input (placeholder: "Salt value...")
5. Button: "Generate Random Salt"

[Action Area]

6. Button: "Generate Hash"
7. Clear button & trash icon

[Result Area]

8. Read-only result field: "Hash Result"
9. Copy button



REQUIREMENTS

1. The hash value must be generated from the UTF-8 bytes of the input text using the selected algorithm, and displayed as a lowercase hexadecimal string.
2. All six algorithms (MD5, SHA-1, SHA-224, SHA-256, SHA-384, SHA-512) must produce correct hash values. The provided test-vectors.json contains the expected values.
3. The default algorithm selection must be MD5.
4. The salt value input and the Generate Random Salt button must be enabled only when "Use Salt" is checked.

5. When "Use Salt" is checked and a salt value exists, the hash input must be UTF8(inputText + displayedSaltText) — a simple text concatenation. A manually entered salt is treated as a normal text string. 6. The Generate Random Salt button must generate 16 random bytes and display them as 32 lowercase hexadecimal characters in the salt input. 7. If the input text is empty, an error notification banner must be displayed. 8. The clear button must reset the input text, the salt value, and the result. 9. The hash result must be displayed in a read-only field, and the Copy button must be displayed only when a result exists.	

4. Security Tab – File Hash Calculator

1. Description:

1.1 The File Hash Calculator computes the MD5, SHA-1, SHA-256, and SHA-512 hash values of a selected file at once.

2. Requirements

ELEMENTS

[File Selection Area]

1. File drop zone with dashed border & document icon
2. Button: "Select File"
3. Selected file name text ("No file selected" when empty)

[File Information Area]

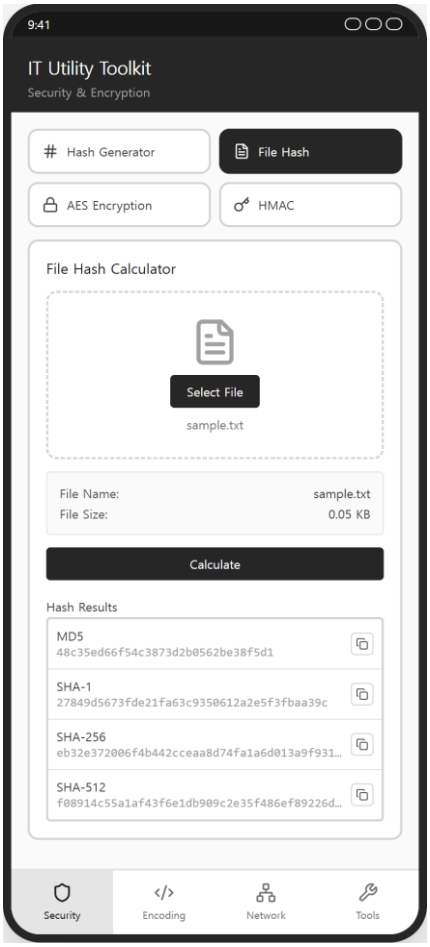
4. "File Name" label & value
5. "File Size" label & value (KB)

[Action Area]

6. Button: "Calculate"

[Result Area]

7. Hash result list: MD5, SHA-1, SHA-256, SHA-512
8. Copy button per hash value



REQUIREMENTS

1. The user must be able to select a file from the device.
2. After selection, the file name and the file size must be displayed. The file size in KB is calculated as fileSizeInBytes / 1024.0 and rounded to two decimal places.
3. When Calculate is tapped, all four hash values (MD5, SHA-1, SHA-256, SHA-512) must be calculated from the original binary file bytes and displayed as lowercase hexadecimal strings.
4. The hash values for the provided sample.txt file must match the values in test-vectors.json.

5. Selecting another file must clear all previous hash results. Cancelling file selection must keep the previous state unchanged. 6. If no file is selected, an error notification banner must be displayed. 7. Each hash value must be displayed in a single line and may be visually truncated, but the Copy button must always copy the complete hash value. 8. A Copy button must be displayed next to each calculated hash value only after calculation.	

5. Security Tab – AES Encryption

1. Description:

1.1 The AES Encryption tool encrypts and decrypts text using an AES symmetric key.

1.2 The user can select the key size (AES-128/192/256) and the cipher mode (CBC/ECB). CBC mode uses a user-entered IV.

2. Requirements

ELEMENTS

[Input Area]

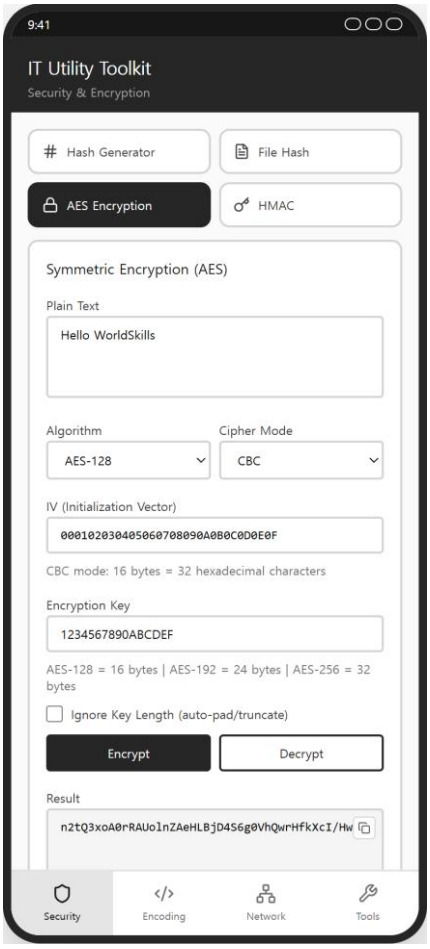
1. Plain text area (placeholder: "Enter text to encrypt/decrypt...")
2. Algorithm dropdown: AES-128, AES-192, AES-256
3. Cipher mode dropdown: CBC, ECB
4. IV input: "IV (Initialization Vector)" (32 hexadecimal characters, monospace)
5. IV hint text: "CBC mode: 16 bytes = 32 hexadecimal characters"
6. Encryption key input
7. Key length hint text: "AES-128 = 16 bytes | AES-192 = 24 bytes | AES-256 = 32 bytes"
8. Checkbox: "Ignore Key Length (auto-pad/truncate)"

[Action Area]

9. Button: "Encrypt" (filled)
10. Button: "Decrypt" (outlined)

[Result Area]

11. Read-only result field: "Result"
12. Copy button



REQUIREMENTS

1. The plain text must be converted to UTF-8 bytes and encrypted with the selected AES key size and cipher mode using PKCS7 padding. (The PKCS5Padding transformation name on Android is acceptable when the results match.)

2. The ciphertext must be displayed as standard Base64 without line breaks. 3. The default selections must be AES-128 and CBC mode. 4. The IV input must be visible and enabled only when CBC mode is selected. ECB mode must not use an IV. 5. The IV must be exactly 16 bytes, entered as 32 hexadecimal characters. An invalid IV must display an error notification banner. 6. CBC encryption and decryption must use the IV entered in the IV field. With the same algorithm, mode, key, and IV, the ciphertext must match the value in test-vectors.json. 7. Decrypt must read a Base64 ciphertext from the input field and restore the original text. (For a round-trip test, the encrypted result may be copied into the input field before tapping Decrypt.) 8. The encryption key may contain printable ASCII characters only, and the key length is measured in bytes. 9. When "Ignore Key Length" is unchecked and the key length does not match the selected key size, an error notification banner must be displayed (e.g., "Key must be exactly 16 bytes for AES-128"). 10. When "Ignore Key Length" is checked, a short key must be padded with ASCII "0" (0x30) characters and a long key must be truncated by bytes. 11. If the text or the key is empty, or decryption fails because of an invalid key, IV, or ciphertext, an error notification banner must be displayed. 12. The result must be displayed in a read-only field, and the Copy button must be displayed only when a result exists.	

6. Security Tab – HMAC Generator

1. Description:

1.1 The HMAC Generator creates a keyed-hash message authentication code from an input text and a secret key.

2. Requirements

ELEMENTS

[Input Area]

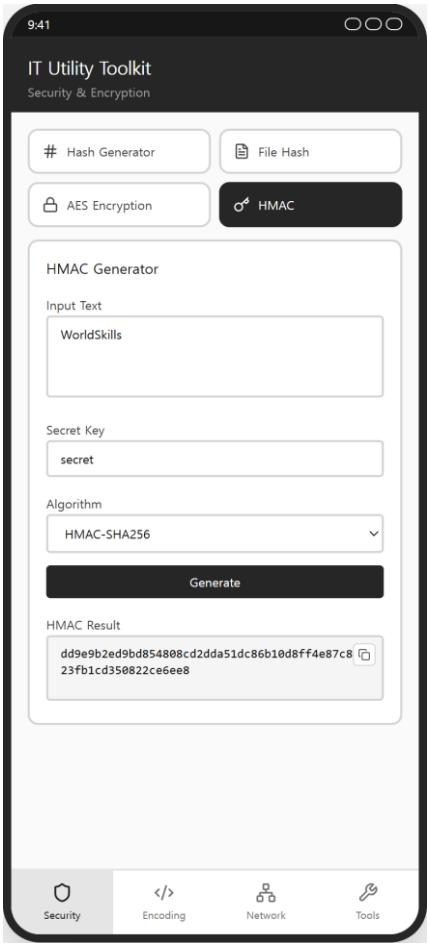
1. Input text area (placeholder: "Enter text...")
2. Secret key input (placeholder: "Enter secret key...")
3. Algorithm dropdown: HMAC-MD5, HMAC-SHA1, HMAC-SHA256, HMAC-SHA512

[Action Area]

4. Button: "Generate"

[Result Area]

5. Read-only result field: "HMAC Result"
6. Copy button



REQUIREMENTS

1. The HMAC value must be computed from the UTF-8 bytes of the input text and the secret key using the selected algorithm, and displayed as a lowercase hexadecimal string.
2. All four algorithms (HMAC-MD5, HMAC-SHA1, HMAC-SHA256, HMAC-SHA512) must produce correct values. The provided test-vectors.json contains the expected values.
3. The default algorithm selection must be HMAC-MD5.
4. If the text or the secret key is empty, an error notification banner must be displayed.
5. The result must be displayed in a read-only field, and the Copy button must be displayed only when a result exists.

7. Encoding Tab – Base64 & URL Encoder/Decoder

1. Description:

1.1 The Base64 tool encodes text to Base64 and decodes Base64 back to text.

1.2 The URL Encoding tool percent-encodes and decodes URLs or text.

2. Requirements

ELEMENTS

[Base64 Card]

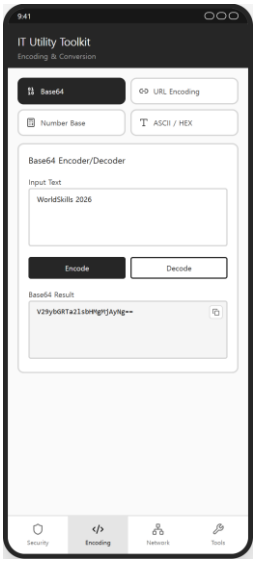
1. Input text area (placeholder: "Enter text to encode/decode...")
2. Button: "Encode" (filled)
3. Button: "Decode" (outlined)
4. Read-only result field: "Base64 Result" & Copy button

[URL Encoding Card]

5. Input text area (placeholder: "Enter URL or text to encode/decode...")
6. Button: "Encode" (filled)
7. Button: "Decode" (outlined)
8. Read-only result field: "URL Encoded Result" & Copy button

REQUIREMENTS

1. Base64 encoding must convert the UTF-8 bytes of the input to standard Base64 (not Base64URL) without line breaks, and decoding must restore the original text (see test-vectors.json).
2. URL encoding must follow RFC 3986: unreserved characters (A-Z, a-z, 0-9, "-", ".", "_", "~") remain unchanged, all other bytes are percent-encoded using UTF-8, spaces are encoded as "%20" (not "+"), and percent-encoded hexadecimal digits are uppercase.
3. The characters "!", "*", "'", "(", ")" may be either percent-encoded or left unchanged; both results are accepted.
4. An Encode → Decode round-trip must restore the original input for both tools.



5. If the input is empty, an error notification banner must be displayed. 6. If decoding fails because of invalid Base64, invalid percent-encoding, or invalid UTF-8, an error notification banner must be displayed. 7. Results must be displayed in read-only fields, and Copy buttons must be displayed only when a result exists.	

8. Encoding Tab – Number Base & ASCII/HEX Converter

1. Description:

1.1 The Number Base Converter converts a number between binary, octal, decimal, and hexadecimal.

1.2 The ASCII/HEX Converter converts text to hexadecimal byte codes and back.

2. Requirements

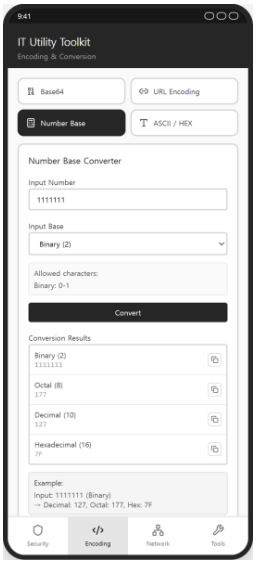
ELEMENTS

[Number Base Card]

1. Number input field
2. Input base dropdown: Binary (2), Octal (8), Decimal (10), Hexadecimal (16)
3. Allowed-characters guide box (changes by selected base)
4. Button: "Convert"
5. Result list: Binary, Octal, Decimal, Hexadecimal & Copy button per value
6. Example guide box

[ASCII/HEX Card]

7. Mode toggle buttons: "Text → Hex", "Hex → Text"
8. Input text area (placeholder changes by mode)
9. Button: "Convert"
10. Read-only output field & Copy button



REQUIREMENTS

1. The input must be validated against the selected base (Binary: 0-1, Octal: 0-7, Decimal: 0-9, Hexadecimal: 0-9/A-F/a-f). Invalid input must display an error notification banner.
2. Only unsigned integers from 0 to 4,294,967,295 are supported. Negative values are not supported, and values outside the supported range must display an error notification banner.
3. The number must be converted to all four bases at once, and hexadecimal results must be displayed in uppercase. Leading

zeros are accepted in the input but are not preserved in the results. 4. The default input base must be Decimal (10), and the allowed-characters guide box must change according to the selected input base. 5. The ASCII/HEX Converter accepts only ASCII characters (0x00-0x7F). Non-ASCII input must display an error notification banner. 6. Text → Hex must convert each character to a two-digit uppercase hexadecimal code separated by spaces (e.g., "Hello" → "48 65 6C 6C 6F"). 7. Hex → Text must convert space-separated hexadecimal byte codes back to text. 8. The default mode must be Text → Hex, and the input placeholder must change according to the selected conversion mode. 9. If the input is empty, an error notification banner must be displayed.	

9. Network Tab – CIDR & IP Address Calculator

1. Description:

1.1 The CIDR Calculator computes network information from an IP address and a CIDR prefix.

1.2 The IP Address Calculator adds or subtracts a number to/from an IP address, or counts the addresses between two IPs.

2. Requirements

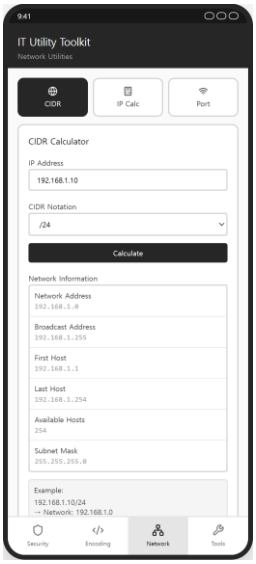
ELEMENTS

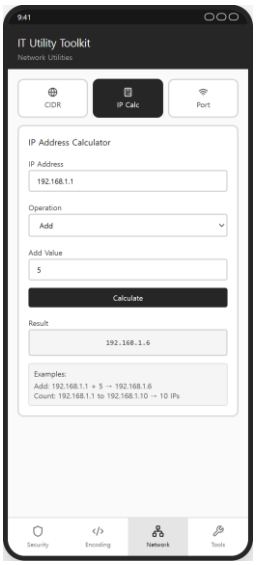
[CIDR Calculator Card]

1. IP address input (placeholder: "192.168.1.10")
2. CIDR notation dropdown: /8, /16, /24, /25, /26, /27, /28, /29, /30
3. Button: "Calculate"
4. Result list: Network Address, Broadcast Address, First Host, Last Host, Available Hosts, Subnet Mask
5. Example guide box

[IP Address Calculator Card]

6. IP address input (placeholder: "192.168.1.1")
7. Operation dropdown: Add, Subtract, Count
8. Conditional input: value input (Add/Subtract) or end IP address input (Count)
9. Button: "Calculate"
10. Result box
11. Example guide box





REQUIREMENTS

1. Only IPv4 addresses and the CIDR prefixes listed in the dropdown are required. IP addresses are treated as unsigned 32-bit values.
2. All six CIDR results must be calculated correctly using bitwise operations (Network = IP AND Mask, Broadcast = Network OR NOT Mask, First Host = Network + 1, Last Host =

Broadcast - 1, Available Hosts = 2^(32-prefix) - 2). The provided test-vectors.json contains the expected values. 3. The default CIDR prefix must be /24, and the default operation must be Add. 4. An invalid IP address (wrong format or an octet outside 0-255) must display an error notification banner. 5. Add and Subtract values must be non-negative integers, and the calculation must carry across octet boundaries (e.g., 192.168.1.250 + 10 = 192.168.2.4). 6. Overflow above 255.255.255.255 or underflow below 0.0.0.0 must display an error notification banner. 7. For Count, the End IP must be greater than or equal to the Start IP, and the result is the inclusive number of IP addresses (e.g., 192.168.1.1 to 192.168.1.10 → 10 IP addresses). 8. The conditional input field must change according to the selected operation.	

10. Network Tab – Port Checker (Wireframe)

1. Description:

1.1 The Port Checker is provided as a wireframe UI only. Real TCP port checking is not required.

2. Requirements

ELEMENTS

[Notice Area]

1. Warning notice box: "Note: Wireframe only. No actual network implementation."

[Input Area]

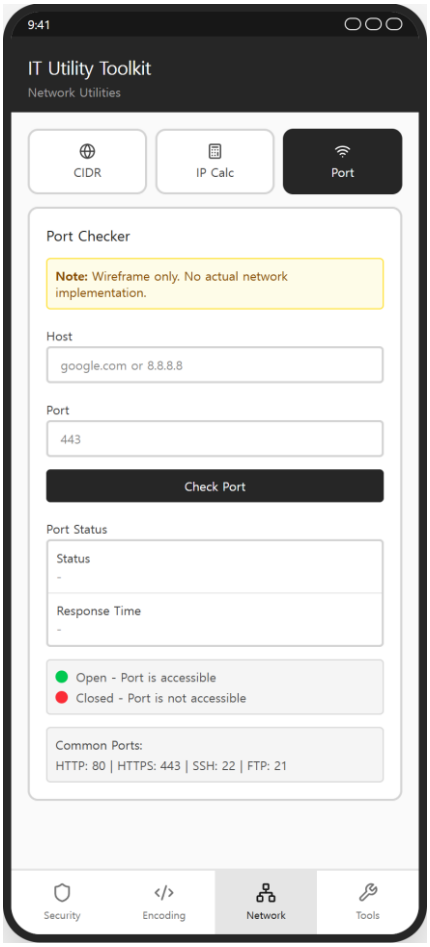
2. Host input (placeholder: "google.com or 8.8.8.8")
3. Port number input (placeholder: "443")
4. Button: "Check Port"

[Result Area]

5. "Status" row & "Response Time" row (values displayed as "-")
6. Legend: green dot = Open, red dot = Closed
7. Common ports guide box: "HTTP: 80 | HTTPS: 443 | SSH: 22 | FTP: 21"

REQUIREMENTS

1. All listed elements must be displayed according to the prototype.
2. No real network communication is required for this tool.
3. When Check Port is tapped, the notification "This feature is a wireframe only." must be displayed, and Status and Response Time must remain "-".



11. Tools Tab – Password & UUID Generator

1. Description:

1.1 The Password Generator creates a random password according to the selected length and character options.

1.2 The UUID Generator creates an RFC 4122 version 4 UUID.

2. Requirements

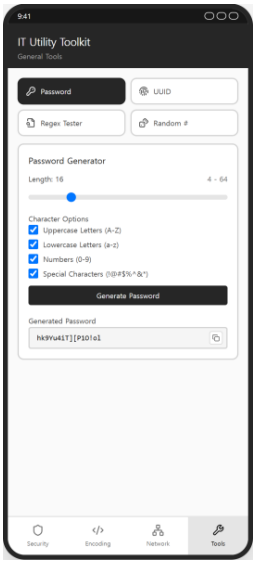
ELEMENTS

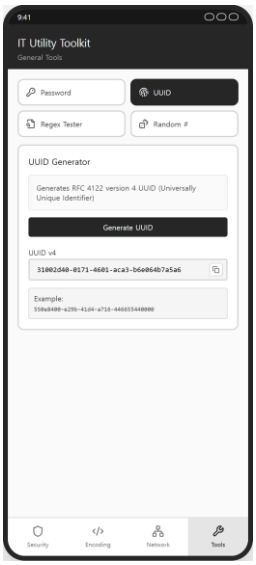
[Password Generator Card]

1. Length slider (4-64) with current value label
2. Checkbox: "Uppercase Letters (A-Z)"
3. Checkbox: "Lowercase Letters (a-z)"
4. Checkbox: "Numbers (0-9)"
5. Checkbox: "Special Characters (!@#$%^&*)"
6. Button: "Generate Password"
7. Read-only result field: "Generated Password" & Copy button

[UUID Generator Card]

8. Information guide box (RFC 4122 version 4)
9. Button: "Generate UUID"
10. Read-only result field: "UUID v4" (placeholder: "xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx") & Copy button
11. Example guide box





REQUIREMENTS

1. The default state must be: length 16 and all four character options checked.
2. The generated password length must match the slider value, and the current slider value must be displayed.
3. The generated password must contain only characters from the selected character sets, and at least one character from each selected set.
4. The exact character sets are: A-Z, a-z, 0-9, and "!@#$%^&*()_+-=[]{}|;:,.<>?".

5. If no character option is selected, an error notification banner must be displayed. 6. The generated UUID must follow the RFC 4122 version 4 format in lowercase (xxxxxxxx-xxxx-4xxx-[89ab]xxx-xxxxxxxxxxxx): the version nibble is "4" and the variant nibble is one of "8", "9", "a", "b". 7. Consecutive UUID generations must produce different values. 8. Results must be displayed in read-only fields, and Copy buttons must be displayed only when a result exists.	

12. Tools Tab – Regex Tester & Random Number Generator

1. Description:

1.1 The Regular Expression Tester checks whether a test string matches a regular expression pattern.

1.2 The Random Number Generator creates one or more random integers within a given range.

2. Requirements

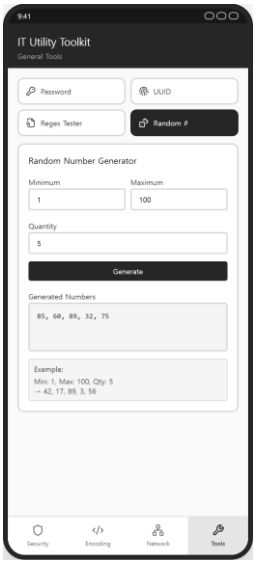
ELEMENTS

[Regex Tester Card]

1. Pattern input (monospace, placeholder: "^[a-zA-Z0-9_]{6,16}$")
2. Test string text area (placeholder: "Enter text to test against the pattern...")
3. Button: "Test"
4. Result box: "✓ Matched" / "✗ Not Matched" / "-"
5. Legend: green dot = Matched, red dot = Not Matched
6. Common patterns guide box (Email, URL, Phone)

[Random Number Generator Card]

7. Number input: "Minimum" (default: 1)
8. Number input: "Maximum" (default: 100)
9. Number input: "Quantity" (default: 1)
10. Button: "Generate"
11. Result box
12. Example guide box



REQUIREMENTS

1. The Regex Tester performs a partial search: the result is Matched when the test string contains at least one match of the pattern. Default regular expression options are used (no flags).
2. The result box must visually change according to the state: matched (green), not matched (red), default (neutral).

3. An invalid regular expression pattern must display an error notification banner. 4. If the pattern or the test string is empty, an error notification banner must be displayed. 5. All generated random values must be independent integers within the inclusive range [Minimum, Maximum], and the number of values must equal Quantity. Negative minimum and maximum values are allowed, and duplicate values are allowed. 6. If Minimum is not less than Maximum, an error notification banner must be displayed. 7. If Quantity is not between 1 and 1000, an error notification banner must be displayed. 8. Generated numbers must be displayed joined by a comma and a space (e.g., "42, 17, 89, 3, 56").	

Instructions to the Competitor

1.1 Android: Competitors must build the final APK file and place it in the root directory of the source project. ※ Final file name: Module_A_{XX}.apk (where XX represents the competitor's country code)

1.2 iOS: Competitors must place the final .app bundle in the root directory of the source project. ※ Final file name: Module_A_{XX}.app (where XX represents the competitor's country code)

1.3 The project source directory must be pushed to the provided VCS (GitHub) repository. (Repository name: mod1_{XX} // XX is the country code)

1.4 Only the materials submitted within the given competition time will be evaluated. (Submission must also be completed within the competition time)

1.5 The Android APK will be evaluated on the specified Android emulator, and the iOS app bundle will be evaluated on the specified iOS Simulator. (Execution may be performed if necessary)